ANSIBLE



¿QUÉ ES ANSIBLE?

Ansible es una potente herramienta de automatización de TI de código abierto que permite gestionar la configuración, el aprovisionamiento, el despliegue de aplicaciones y la orquestación de servicios de forma sencilla, segura y escalable.

Los departamentos de TI tienen que hacer frente a tareas repetitivas que acaban consumiendo un tiempo valioso al personal cualificado. Esto a menudo supone dejar de hacer otras tareas de valor añadido por falta de tiempo.

A través de la automatización de tareas, se obtienen los siguientes beneficios:



CLASIFICACIÓN Y PAPEL

Dentro del ecosistema DevOps y de Infraestructura como Código (IaC),
Ansible se clasifica bajo los siguientes pilares:

Gestión de Configuración : Se encarga de definir y mantener los servidores, aplicaciones y redes en un "estado deseado".

Herramienta Basada en Modelo Push: El nodo de control inicia activamente la comunicación y "empuja" las configuraciones hacia los servidores objetivo.

Arquitectura Sin Agentes: No requiere instalar ningún software residente en los nodos gestionados. Se comunica utilizando canales estándar como SSH para Linux o WinRM para Windows.

ARQUITECTURA

Ansible sigue una arquitectura sencilla basada en push donde un nodo de control gestiona múltiples hosts objetivo sin requerir agentes en sistemas remotos.

Componentes principales

Nodo de control: La máquina desde la que se instala Ansible y desde donde se ejecutan los playbooks. Se conecta a nodos gestionados vía SSH y envía configuraciones.

Nodos gestionados: Servidores objetivo que configura Ansible. No es necesario instalar software Ansible en estos hosts: en Linux, solo necesitan acceso a SSH y Python.

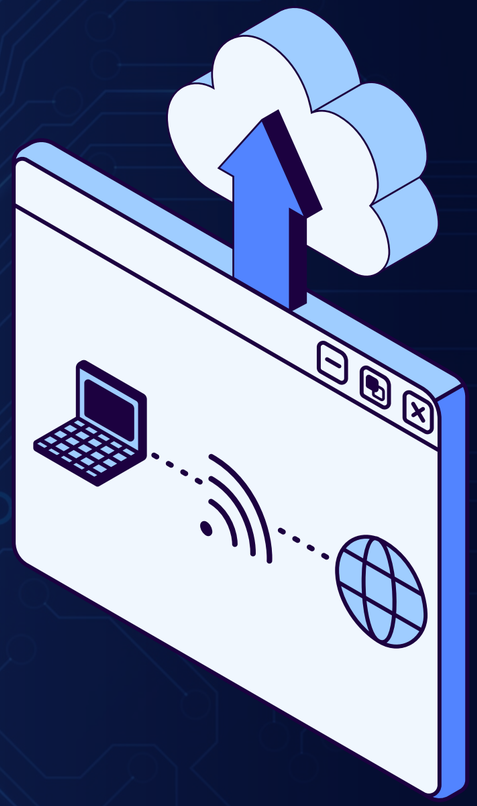
Inventario: Archivo o directorio que contiene información sobre hosts gestionados, incluyendo direcciones IP, detalles de conexión y agrupaciones de hosts.

Módulos: scripts Python que realizan tareas específicas . Los módulos se envían a nodos gestionados, se ejecutan y luego se eliminan.

Playbooks: archivos YAML que contienen listas ordenadas de tareas a ejecutar en hosts gestionados. Definen el estado deseado de tu infraestructura.

CONCEPTOS PRINCIPALES

- **Tarea**: acción que Ansible ejecuta, como instalar un paquete o copiar un archivo.
- **Variable**: dato que permite personalizar la configuración sin duplicar el código.
- **Plantilla**: archivo que utiliza variables para generar contenido o configuraciones.
- **Handler**: tarea especial que se ejecuta cuando otra tarea registra un cambio y la notifica.
- **Idempotencia**: permite repetir la automatización sin realizar cambios innecesarios cuando el estado deseado ya se alcanzó.



EJEMPLO BÁSICO
DE CONFIGURACIÓN

```yaml
---
- name: Configurar servidor web
  hosts: web
  become: true

  tasks:
    - name: Instalar Nginx
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true
        cache_valid_time: 3600

    - name: Iniciar y habilitar Nginx
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
```

name: describe el propósito del playbook o de una tarea.
hosts: web: selecciona los equipos del grupo web, definido en el inventario.
become: true: permite ejecutar las tareas con privilegios administrativos.

tasks: contiene las acciones que se ejecutarán en orden.

apt: actualiza el índice de paquetes (si tiene más de una hora) e instala Nginx si todavía no está instalado.

service: asegura que Nginx esté ejecutándose y que arranque automáticamente con el sistema.

FLUJO DE TRABAJO

Definir 01.

El desarrollador escribe el inventario, las variables y el playbook.

Versionar 02.

Los archivos se guardan en Git para revisar y registrar los cambios.

Ejecutar 03.

Ansible lee el inventario y realiza las tareas sobre los servidores seleccionados.

Configurar 04.

Los módulos instalan paquetes, generan archivos y administran servicios.

Verificar 05.

Se comprueba que la aplicación responde y que la versión desplegada es correcta.

INTEGRACIÓN CON TECNOLOGÍAS

Git y GitHub: permiten almacenar, versionar y revisar los playbooks y archivos de configuración.

Servidores Linux: administra paquetes, usuarios, permisos y servicios mediante conexiones SSH.

Docker y Docker Compose: Ansible puede preparar el servidor, distribuir archivos y automatizar el despliegue de contenedores.

CI/CD: Jenkins o GitHub Actions pueden ejecutar Ansible después de las pruebas para configurar el entorno y desplegar una nueva versión.

VENTAJAS

01. Reducción de errores humanos

02. Eficiencia y ahorro de tiempo

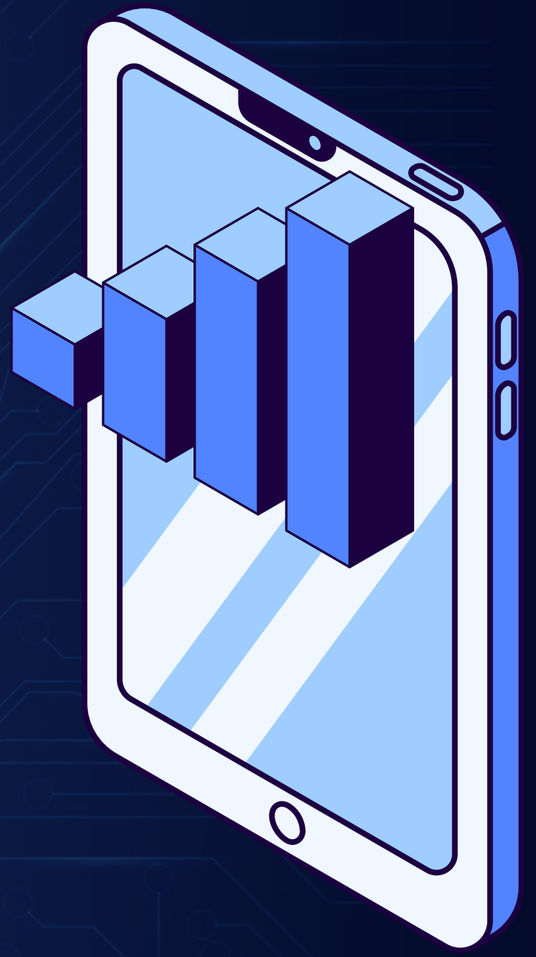
03. Facilita la colaboración y estandarización

LIMITACIONES

01. Lento en escalas masivas.

02. Gestión de estado débil.

DEMOSTRACIÓN **PRÁCTICA**



CONCLUSIÓN

¿Qué automatiza? 01.

La instalación de paquetes, la configuración de servidores, la distribución de archivos y la administración de servicios.

¿Qué reemplaza? 02.

Los pasos manuales repetitivos que una persona realizaría en cada servidor.

¿Cómo participa en CI/CD? 03.

El pipeline ejecuta Ansible para preparar el entorno, desplegar una versión y comprobar su funcionamiento.

¿Qué aporta? 04.

Un proceso repetible y versionado que reduce diferencias entre entornos y facilita revisar los cambios.


GRACIAS